

- Description

This module manages a smart elevator scheduling and load balancing system for high-rise buildings, ensuring that:

1. **Each elevator's passenger load is continuously monitored to prevent overloading.**
2. **If an elevator reaches its maximum load, new passengers are redirected to less crowded elevators.**
3. **Elevator travel is optimized based on real-time demand to reduce wait times and energy consumption.**
4. **A log of all elevator movements, load statuses, and efficiency optimizations is maintained for performance analysis.**

This system helps improve elevator efficiency, reduce congestion, and enhance passenger experience.

- Specification

module SmartElevatorScheduling

var currentLoad, maxCapacity, currentFloor, targetFloor: integer

var log: sequence of string

process enterElevator(numPassengers: integer)

functional scenario form:

currentLoad + numPassengers <= maxCapacity && currentLoad = currentLoad + numPassengers, log = log $\oplus$ ("Passengers Entered: " + numPassengers + " | Load: " + currentLoad + "/" + maxCapacity), Passengers entered successfully.

||

currentLoad + numPassengers > maxCapacity && log = log $\oplus$ ("Overload Warning: Redirecting passengers."), Overload warning triggered, passengers redirected.

end process

process exitElevator(numPassengers: integer)

functional scenario form:

currentLoad = max(0, currentLoad - numPassengers), log = log $\oplus$ ("Passengers Exited: " + numPassengers + " | Load: " + currentLoad + "/" + maxCapacity), Passengers exited, load updated.

end process

process moveElevator(targetFloor: integer)

functional scenario form:

(targetFloor $\neq$ currentFloor) && log = log $\oplus$ ("Moved from Floor " + currentFloor + " to Floor " + targetFloor), currentFloor = targetFloor, Elevator moved successfully.

OR

(targetFloor = currentFloor) && log = log $\oplus$ ("Elevator already at Floor " + targetFloor), No movement needed.

end process

end module

- Java Code

```
package code;
import java.util.*;
class Elevator45 {
    private String id;
    private int maxCapacity;
    private int currentLoad;
    private int currentFloor;
    private List<String> log;
    public Elevator45(String id, int maxCapacity) {
        this.id = id;
        this.maxCapacity = maxCapacity;
        this.currentLoad = 0;
        this.currentFloor = 1;
        this.log = new ArrayList<>();
    }
    public boolean enterElevator(int numPassengers) {
        if (currentLoad + numPassengers > maxCapacity) {
            System.out.printf("Elevator %s Overloaded! Redirecting Passengers.%n", id);
            log.add("Overload Warning: Redirecting passengers.");
            return false;
        }
        currentLoad += numPassengers;
        System.out.printf("Elevator %s: %d Passengers Entered | Current Load: %d/%d%n", id, numPassengers, currentLoad, maxCapacity);
        log.add(String.format("Passengers Entered: %d | Load: %d/%d", numPassengers, currentLoad, maxCapacity));
        return true;
    }
    public void exitElevator(int numPassengers) {
        currentLoad = Math.max(0, currentLoad - numPassengers);
        System.out.printf("Elevator %s: %d Passengers Exited | Remaining Load: %d/%d%n", id, numPassengers, currentLoad, maxCapacity);
        log.add(String.format("Passengers Exited: %d | Load: %d/%d", numPassengers, currentLoad, maxCapacity));
    }
    public void moveElevator(int targetFloor) {
        if (targetFloor == currentFloor) {
            System.out.printf("Elevator %s is already on Floor %d.%n", id, targetFloor);
```

```

        return;
    }
    System.out.printf("Elevator %s Moving: Floor %d → Floor %d\n",
id, currentFloor, targetFloor);
    log.add(String.format("Moved from Floor %d to Floor %d", currentFloor, targetFloor));
    currentFloor = targetFloor;
}
public void printLog() {
    System.out.printf("\nElevator %s Operation Log:\n", id);
    for (String logEntry : log) {
        System.out.println(logEntry);
    }
}
}
public class c45_SmartElevatorSystem {
    public static void main(String[] args) {
        Elevator45 elevatorA = new Elevator45("A", 10);
        // testcase-VT:
        //     elevatorA.enterElevator(5);
        //     elevatorA.enterElevator(7);
        //     elevatorA.moveElevator(5);
        //     elevatorA.exitElevator(3);
        //     elevatorA.moveElevator(1);
        //     elevatorA.moveElevator(1);

        // testcase-VT:
        //     elevatorA.enterElevator(83886080);
        //     elevatorA.enterElevator(327727);
        //     elevatorA.moveElevator(47);
        //     elevatorA.exitElevator(5066061);
        //     elevatorA.moveElevator(5);
        //     elevatorA.moveElevator(8830);

        // testcase-Z3:
        //     elevatorA.enterElevator(5);
        //     elevatorA.enterElevator(7);
        //     elevatorA.enterElevator(7);
        //     elevatorA.exitElevator(3);
        //     elevatorA.enterElevator(7);
        //     elevatorA.enterElevator(7);

        // testcase-UVT:
        elevatorA.enterElevator(5);
    }
}

```

```

        elevatorA.enterElevator(7);
        elevatorA.moveElevator(5);
        elevatorA.exitElevator(3);
        elevatorA.moveElevator(1);
        elevatorA.moveElevator(1);
//        elevatorA.printLog();
    }
}

```

- Coverage report

1. Test case number: 6
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.7 %	6	836	842
✓ src	0.7 %	6	836	842
✓ code	0.7 %	6	836	842
✓ c45_SmartElevatorSystem.java	100.0 %	6	0	6
> c45_SmartElevatorSystem		0	0	0
> Elevator45	100.0 %	6	0	6

3. Vibration testing instruction coverage: 99%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	1.4 %	290	21,076	21,366
✓ src	1.4 %	290	21,076	21,366
✓ code	1.4 %	290	21,076	21,366
✓ c45_SmartElevatorSystem.java	99.0 %	290	3	293
> c45_SmartElevatorSystem	90.6 %	29	3	32
> Elevator45	100.0 %	261	0	261

4. Fuzz testing branch coverage: 66.7%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.5 %	4	838	842
✓ src	0.5 %	4	838	842
✓ code	0.5 %	4	838	842
✓ c45_SmartElevatorSystem.java	66.7 %	4	2	6
> c45_SmartElevatorSystem		0	0	0
> Elevator45	66.7 %	4	2	6

5. Fuzz testing instruction coverage: 72.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	1.0 %	212	21,154	21,366
✓ src	1.0 %	212	21,154	21,366
✓ code	1.0 %	212	21,154	21,366
✓ c45_SmartElevatorSystem.java	72.4 %	212	81	293
> c45_SmartElevatorSystem	90.6 %	29	3	32
> Elevator45	70.1 %	183	78	261

6. Z3 branch coverage: 33.3%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.2 %	2	838	840
✓ src	0.2 %	2	838	840
✓ code	0.2 %	2	838	840
✓ c45_SmartElevatorSystem.java	33.3 %	2	4	6
> Elevator45	33.3 %	2	4	6
> c45_SmartElevatorSystem		0	0	0

7. Z3 instruction coverage: 67.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.9 %	198	22,479	22,677
▼ src	0.9 %	198	22,479	22,677
▼ code	0.9 %	198	22,479	22,677
▼ c45_SmartElevatorSystem.java	67.3 %	198	96	294
> Elevator45	64.4 %	168	93	261
> c45_SmartElevatorSystem	90.9 %	30	3	33

8. Unimproved V-method branch coverage: 66.7%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.5 %	4	836	840
▼ src	0.5 %	4	836	840
▼ code	0.5 %	4	836	840
▼ c45_SmartElevatorSystem.java	66.7 %	4	2	6
> Elevator45	66.7 %	4	2	6
> c45_SmartElevatorSystem		0	0	0

9. Unimproved V-method instruction coverage: 89.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.2 %	261	22,413	22,674
▼ src	1.2 %	261	22,413	22,674
▼ code	1.2 %	261	22,413	22,674
▼ c45_SmartElevatorSystem.java	89.7 %	261	30	291
> Elevator45	89.7 %	234	27	261
> c45_SmartElevatorSystem	90.0 %	27	3	30